

LEADERBOARD **MANAGEMENT SYSTEM** **USING RED-BLACK TREE**

Master of Computer Applications
School of Computer & Information Sciences
University of Hyderabad, Hyderabad - 500046

Submitted to
Dr.Arun Kumar Das

Submitted by
Satyabrata Rath - 25mcmc36
Ayush Toppo - 25mcmc22
B.Rikheet Kumar - 25mcmc24

CANDIDATE’S DECLARATION

We, Satyabrata Rath , Ayush Toppo , B.Rikheet Kumar are hereby certify that the work presented in this report, entitled “Leaderboard Management System using Red Black Tree” is an authentic record of our own work carried out as part of an academic assignment, at the School of Computer and Information Sciences, University of Hyderabad.

The matter presented in this report has not been submitted elsewhere for the award of any degree or diploma from any institution.

We declare that we have properly cited all references for any text(s) or equation(s) that have been taken from other sources. We further affirm that we have not willfully copied any part of another person's work, including text, data, results, or findings from journals, books, websites, reports, dissertations, or theses, and presented them as our own without proper acknowledgment.

Date: 30/04/2026

Signature of the Candidate

TABLE OF CONTENTS

1. **Abstract**
2. **Introduction**
3. **System Design Objectives**
4. **Data Structures Used**
 - 4.1 Player Data Structure (Payload)
 - 4.2 Red-Black Tree Node Structure
 - 4.3 NIL Sentinel Node & Memory Layout
5. **Red-Black Tree Implementation Details**
 - 5.1 Properties Enforced
 - 5.2 Rotation Logic (Left & Right)
 - 5.3 Insertion & Deletion Fix-Up Cases
6. **Core Algorithms & Traversal**
 - 6.1 Player Insertion & Score-Based Ordering
 - 6.2 Player Deletion & Replacement Logic
 - 6.3 Reverse In-Order Traversal (Ranking Logic)
 - 6.4 Search & Range Query Implementation
7. **Program Workflow & Module Description**
 - 7.1 Function Call Graph
 - 7.2 Global Variables & Constants
8. **Validation & Error Handling**
 - 8.1 Input validation & handling
 - 8.2 Duplicate ID Prevention
 - 8.3 Memory Safety & Deallocation
9. **File Storage Mechanism**
 - 9.1 Binary File Storage Format
 - 9.2 Save and Load Logic
10. **User Interface Menu**
11. **Testing & Results**
 - 11.1 Test Case Matrix
 - 11.2 Performance & Complexity Analysis
12. **Challenges & Solutions**

13. Limitations, Conclusion & Future Enhancements

14. References

1. Abstract

This report presents the design and implementation of a Leaderboard Management System developed in the C programming language. The system uses a Red-Black Tree, a self-balancing binary search tree, to efficiently manage player records based on their scores. By maintaining balance through rotations and color properties, the structure guarantees $O(\log n)$ time complexity for insertion, deletion, and search operations.

Each player is represented by an ID, name, and score, and the leaderboard dynamically ranks players using a reverse in-order traversal, ensuring that higher scores appear at the top. The system supports key operations such as validated insertion, deletion, search with rank display, and range queries based on rank and score.

To ensure reliability, the application includes input validation (such as score limits and duplicate ID checks) and proper memory management. Additionally, a binary file mechanism is implemented to provide persistent storage, allowing data to be saved and reloaded across sessions.

Overall, the project demonstrates how Red-Black Trees can be effectively applied in real-world scenarios requiring efficient data organization, dynamic ranking, and fast retrieval, making it suitable for leaderboard and ranking-based applications.

2.Introduction

Leaderboard systems are widely used in applications such as online gaming, competitive platforms, and performance tracking systems where ranking of users based on scores is essential. These systems require efficient handling of dynamic data, where players can be frequently added, removed, or updated while maintaining a sorted order.

Traditional data structures like arrays or linked lists can be used to store such data, but they suffer from inefficient performance for large datasets, particularly due to $O(n)$ time complexity for search and update operations. This limitation makes them unsuitable for real-time ranking systems.

To address this problem, this project implements a Leaderboard Management System using a Red-Black Tree, a self-balancing binary search tree that ensures $O(\log n)$ time complexity for insertion, deletion, and search operations. The balancing is maintained using specific properties and rotation techniques, allowing the system to remain efficient even with continuous modifications.

The application is developed in C and manages player records consisting of ID, name, and score. The leaderboard is maintained in descending order of scores, and rankings are generated dynamically using reverse in-order traversal.

The system includes functionalities such as inserting players with validation, deleting records, searching players along with their rank, displaying the leaderboard, and filtering players based on rank and score ranges. Additionally, a binary file mechanism is implemented to provide persistent storage, allowing data to be saved and loaded across program executions.

This project demonstrates the practical use of Red-Black Trees in building efficient and scalable ranking systems, highlighting their importance in real-world applications requiring fast data access and dynamic ordering.

3. System Design objectives

The following objectives guided the development of the Leaderboard System

Objective	Description	Achieved
Efficient Data Storage	Store player records consisting of ID, Name, and Score	Yes
Balanced Data Structure	Use a Red-Black Tree to maintain a balanced structure	Yes
Optimized Performance	Ensure $O(\log n)$ time complexity for insertion, deletion, and search operations	Yes
Dynamic Ranking	Maintain leaderboard in descending order of scores	Yes
Rank Calculation	Generate player ranks dynamically using reverse in-order traversal	Yes
Search Functionality	Search players by ID and display rank, name, and score	Yes
Range Queries	Support filtering players by rank range and score range	Yes
Input Validation	Prevent duplicate IDs and enforce valid score input (0–100)	Yes
Data Persistence	Enable saving and loading data using binary file storage (players.dat)	Yes

These objectives ensure that the system is efficient, scalable, and capable of handling dynamic ranking operations effectively using a balanced data structure.

4. Data Structures Used

4.1 Player Data Structure (Payload)

The Player structure stores the actual data of each entry in the leaderboard. It contains an integer id, a character array for the name, and an integer score. The id serves as a unique identifier for each player, ensuring no duplicates are allowed. The name field stores the player's name, and the score represents the player's performance value used for ranking.

This structure acts as the payload of each node in the Red-Black Tree and is the fundamental unit of data stored in the system.

4.2 Red-Black Tree Node Structure

The Node structure represents each node in the Red-Black Tree. It contains a Player data field, a color field (either RED or BLACK), and three pointers: left, right, and parent.

This structure enables efficient traversal and manipulation of the tree. The color property is essential for maintaining the balancing rules of the Red-Black Tree, ensuring that the height of the tree remains logarithmic.

4.3 NIL Sentinel Node

4.3.1 NIL Node

A global NIL node is used as a sentinel to represent all leaf nodes in the Red-Black Tree. It is colored BLACK and shared across the entire tree. Using a NIL sentinel simplifies the implementation of insertion, deletion, and rotation operations. Instead of checking for NULL pointers repeatedly, the program compares pointers against the NIL node, ensuring that every node always has valid left, right, and parent references.

4.3.2 Memory Layout

The tree is organized using a global root pointer that represents the starting point of the Red-Black Tree. Each node dynamically allocates memory and links to its left, right, and parent nodes.

All leaf nodes point to the same NIL sentinel, which eliminates the need for special-case handling of NULL pointers. This design ensures consistency, simplifies tree operations, and improves code reliability.

These data structures form the foundation of the system, enabling efficient storage, balanced operations, and dynamic ranking using the Red-Black Tree.

5. Red-Black Tree Implementation Details

This section describes how the Red-Black Tree is implemented to maintain balance and ensure efficient operations. The implementation follows standard Red-Black Tree rules using colour properties, rotations, and fix-up procedures after insertion and deletion.

5.1 Properties Enforced

- The implementation strictly maintains the five fundamental properties of a Red-Black Tree:
- Every node is either RED or BLACK.
- The root node is always BLACK.
- No two consecutive RED nodes are allowed (a RED node cannot have a RED parent).
- Every path from a node to its descendant NIL nodes contains the same number of BLACK nodes, known as the black-height.
- All NIL (sentinel) nodes are considered BLACK.
- These properties ensure that the tree remains approximately balanced, guaranteeing $O(\log n)$ time complexity for insertion, deletion, and search operations.

5.2 Rotation Logic (Left & Right)

To maintain balance during updates, the implementation uses two types of tree rotations:

Left Rotation

- A left rotation is performed when a node's right subtree becomes heavy.
- The right child of a node becomes its parent
- The original node becomes the left child of its right child
- The left subtree of the new parent is reassigned appropriately
- This rotation helps move larger values upward and restore balance.

Right Rotation

- A right rotation is the mirror operation of a left rotation.
- The left child of a node becomes its parent
- The original node becomes the right child of its left child

- The right subtree of the new parent is reassigned accordingly
- Both rotations:
- Update parent pointers
- Maintain binary search tree ordering
- Execute in $O(1)$ time

5.3 Insertion & Deletion Fix-Up Cases

5.3.1 Insertion Fix-Up

After inserting a new node (always coloured RED), violations of Red-Black properties may occur. A fix-up procedure is applied based on the colour of the node's uncle

Case 1: Uncle is RED

Recolour parent and uncle -> BLACK

Recolour grandparent -> RED

Move upward to continue checking

Case 2: Uncle is BLACK & Node is an Inner Child

Perform a rotation to convert into Case 3

Example: Left-Right or Right-Left case

Case 3: Uncle is BLACK & Node is an Outer Child

Perform a rotation on the grandparent

Recolour nodes appropriately

Tree becomes balanced

Finally, the root is always recoloured BLACK

5.3.2 Deletion Fix-Up

When a BLACK node is deleted, the Red-Black Tree may violate its properties. This introduces a double black (DB) condition. The fix-up process resolves this using multiple cases based on the sibling's colour and its children.

Case 1: Deleted node is RED

Simply remove the node

No violation occurs

Case 2: Deleted node is BLACK with one RED child

Replace node with its child

Recolour child to BLACK

Case 3: Double Black at Root

Remove extra black

Tree becomes valid

Case 4: Sibling is BLACK & both children are BLACK

Recolour sibling -> RED

Move double black to parent

Repeat fix-up

Case 5: Sibling is RED

Swap colours of parent and sibling

Perform rotation toward DB

Convert into another case

Case 6: Sibling is BLACK & at least one RED child

Perform appropriate rotation

Recolour nodes

Remove double black

Fix-up completes

These fix-up procedures ensure that all Red-Black Tree properties are preserved after insertion and deletion, maintaining tree balance and guaranteeing $O(\log n)$ performance.

6. Core Algorithms & Traversal

This section explains the core operations performed on the Red-Black Tree for managing player records and maintaining leaderboard rankings efficiently.

6.1 Player Insertion & Score-Based Ordering

The insertion algorithm begins by creating a new player record containing ID, name, and score. A new node is inserted into the Red-Black Tree following standard Binary Search Tree (BST) insertion rules, where nodes are positioned based on their score.

- If the score is higher, the node is placed toward the right subtree
- If the score is lower, it is placed toward the left subtree

Each newly inserted node is initially coloured RED. After insertion, the insertion fix-up procedure is applied to restore Red-Black Tree properties using recolouring and rotations.

This approach ensures that the leaderboard remains sorted by score while maintaining $O(\log n)$ time complexity.

6.2 Player Deletion & Replacement Logic

The deletion algorithm removes a player node from the Red-Black Tree based on its ID or score.

- If the node has no child, it is removed directly
- If the node has one child, it is replaced by its child
- If the node has two children, it is replaced with its in-order successor

When a BLACK node is deleted, it may violate Red-Black properties. In such cases, the deletion fix-up procedure is applied to restore balance using rotations and recolouring.

This ensures that the tree remains balanced and efficient after deletions.

6.3 Reverse In-Order Traversal (Ranking Logic)

To generate the leaderboard ranking, the system uses reverse in-order traversal.

Traversal order:

Right -> Root -> Left

- This ensures that nodes are visited in descending order of scores
- Players with higher scores appear first (top ranks)
- A rank counter is maintained and incremented during traversal

This method eliminates the need for additional sorting and directly produces a sorted leaderboard efficiently.

6.4 Search & Range Query Implementation

6.4.1 Search Operation

The search algorithm locates a player using their ID or score by traversing the Red-Black Tree.

- At each step, the algorithm compares values and moves left or right accordingly
- Once found, the player's details (ID, name, score, rank) are displayed

The search by score operation runs in $O(\log n)$ time.

6.4.2 Range Query

The system supports filtering players within a given score range or rank range.

- The tree is traversed selectively to include only nodes within the specified range
- Efficient pruning avoids unnecessary traversal of unrelated nodes

This allows fast retrieval of subsets of players without scanning the entire dataset.

These core algorithms and traversal techniques enable efficient data management, dynamic ranking, and fast retrieval operations in the leaderboard system.

7. Program Workflow & Module Description

7.1 Function Call Graph

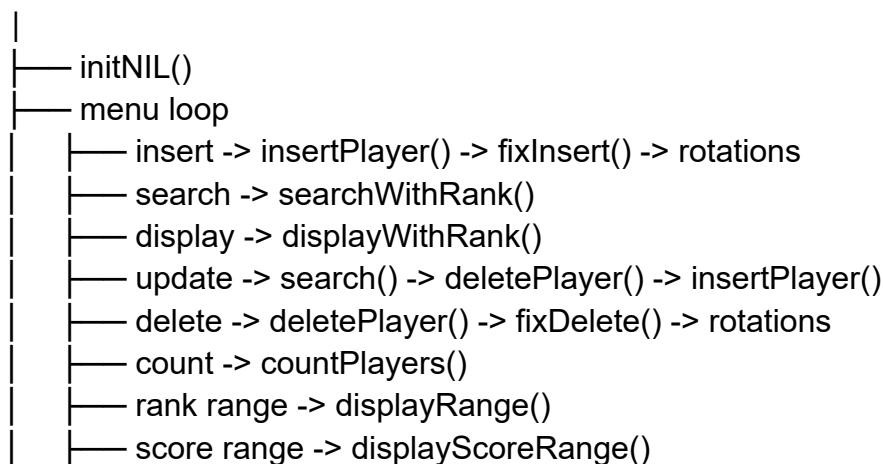
The program execution begins in the `main()` function, where the Red-Black Tree is initialized using `initNIL()` and the root is set to the NIL sentinel node.

The program then enters an infinite menu-driven loop, allowing the user to perform various operations on the Leaderboard Management System. Based on user input, the corresponding functions are invoked:

- Insert Player -> `insertPlayer()` -> `fixInsert()` -> rotations (if needed)
- Search Player -> `searchWithRank()`
- Display Leaderboard -> `displayWithRank()` (reverse in-order traversal)
- Update Score -> `search()` -> `deletePlayer()` -> `insertPlayer()`
- Delete Player -> `deletePlayer()` -> `fixDelete()` -> rotations (if needed)
- Count Players -> `countPlayers()`
- Rank Range Query -> `displayRange()`
- Score Range Query -> `displayScoreRange()`
- Save Data -> `saveFile()` -> `saveAll()`
- Load Data -> `loadFile()` -> `insertPlayer()`
- Clear Data -> `clearAll()` -> `freeTree()`

On program termination, memory is released using `freeTree()` and the NIL node is freed.

`main()`



```

|   |—— save -> saveFile() -> saveAll()
|   |—— load -> loadFile() -> insertPlayer()
|   |—— clear -> clearAll() -> freeTree()
|—— freeTree(), free(NIL)

```

7.2 Global Variables & Constants

The system uses a few important global variables and constants to simplify implementation and ensure consistency:

root

Represents the root of the Red-Black Tree and acts as the entry point for all operations

NIL (Sentinel Node)

A special node used to represent all leaf nodes

It is always colored BLACK and avoids repeated NULL checks

nextId

Maintains the next available unique ID for players during insertion and loading

Color Constants (RED, BLACK)

Used to represent node colors in the Red-Black Tree

These global components help maintain tree structure, simplify edge-case handling, and ensure efficient execution of operations.

8. Validation & Error Handling

8.1 Input Validation & Handling

The system includes input validation to ensure that only correct and meaningful data is accepted from the user.

- **ID Validation**

The ID is first taken as a string and checked to confirm that it contains only digits. If invalid input is detected, the user is asked to re-enter the ID

- **Score Validation**

The score is validated to ensure it is a numeric value between 0 and 100. Any invalid or out-of-range input is rejected

- **Handling Invalid Inputs**

If non-numeric input is entered where a number is expected, the input buffer is cleared using `getchar()` to prevent incorrect values being processed

These checks ensure that invalid data does not enter the system and help maintain consistency.

8.2 Duplicate ID Prevention

Each player must have a unique ID.

- Before inserting a new player, the function `idExists()` is used to check if the ID already exists in the tree
- If a duplicate ID is found, the system displays an error message and asks the user to enter a different ID

This prevents duplicate records and ensures reliable search, update, and deletion operations.

8.3 Memory Safety & Deallocation

The program handles memory carefully to avoid leaks and ensure stable execution.

- Memory for each node is dynamically allocated using `malloc()` during insertion
- The function `freeTree()` is used to recursively free all nodes in the tree
- The `clearAll()` function removes all data, resets the tree, and clears stored records
- Before exiting the program, all allocated memory including the NIL node is freed

This ensures proper memory management and prevents memory-related issues

9. File Storage Mechanism

9.1 Binary File Storage Format

- The system uses a binary file named `players.dat` to store player records persistently.
- Each record in the file corresponds to a `Player` structure, which contains:
 - ID
 - Name
 - Score
- Data is stored using binary format with the `fwrite()` function, which writes the structure directly to the file

- Binary storage is used instead of text format because:
 - It is faster for read and write operations
 - It reduces file size
 - It preserves the exact memory structure of the data

This approach allows efficient storage and retrieval of player data across program executions.

9.2 Save and Load Logic

The system provides functions to save and load data using file operations.

Save Operation

- The function `saveFile()` opens the file `players.dat` in write mode (`wb`)
- The function `saveAll()` performs an in-order traversal of the tree
- Each player record is written to the file using `fwrite()`
- After writing all records, the file is closed

This ensures that all current data in the tree is stored permanently.

Load Operation

- The function `loadFile()` opens the file `players.dat` in read mode (`rb`)
- Each record is read using `fread()` into a `Player` structure
- The record is inserted into the Red-Black Tree using `insertPlayer()`
- The `nextId` value is updated to maintain unique IDs

If the file does not exist, the system displays a message and continues without loading data.

Clear Operation

- The `clearAll()` function deletes all nodes from memory using `freeTree()`
- The file is reset by opening it in write mode (`wb`)
- The `nextId` counter is reset

This allows the system to start fresh without any previously stored data

10. User Interface Menu

The system provides a simple console-based menu interface that allows users to interact with the Leaderboard Management System. The menu is displayed repeatedly, enabling the user to perform multiple operations until they choose to exit.

```
Choice: 12
● PS D:\ds project> gcc leaderboardgemini.c -o a
○ PS D:\ds project> ./a

1.Insert
2.Search
3.Display
4.Update
5.Delete
6.Count
7.Rank Range
8.Score Range
9.Save
10.Load
11.Clear
12.Exit
Choice: █
```

Description of Operations

- **Insert Player**
Adds a new player by taking ID, name, and score as input with proper validation
- **Search Player**
Searches for a player using ID and displays name, score, and rank
- **Display Leaderboard**
Displays all players in descending order of scores along with ranks
- **Update Player Score**
Updates the score of an existing player by deleting and reinserting the node
- **Delete Player**
Removes a player from the system using their ID
- **Count Players**
Displays the total number of players currently stored

- **Rank Range Query**
Displays players within a specified rank range
- **Score Range Query**
Displays players whose scores fall within a given range
- **Save Data**
Saves all player data to a binary file
- **Load Data**
Loads player data from the file into the system
- **Clear All Data**
Deletes all records from memory and resets the system
- **Exit**
Terminates the program after freeing allocated memory

User Interaction Flow

The user selects an option by entering a number

Invalid inputs are handled and the user is prompted again

After each operation, the menu is displayed again

The program continues until the user selects the exit option

11. Testing & Results

11.1 Test Case Matrix

Test Case	Input	Expected Output	Actual Result
Insert valid player	ID=1, Name=RK, Score=90	Player inserted successfully	Pass
Insert duplicate ID	ID=1 again	ID already exists!	Pass
Insert invalid ID	ID = abc	Invalid ID. Only numbers allowed.	Pass
Insert invalid score	Score = 150	Rejected (0–100 only)	Pass
Search existing player	ID=1	Displays name, rank, score	Pass
Search non-existing player	ID=99	Player not found	Pass
Delete existing player	ID=1	Player deleted successfully	Pass

Delete non-existing player	ID=99	Player not found	Pass
Display leaderboard	Multiple players	Sorted by score (descending)	Pass
Rank range query	Range 1–3	Top 3 players displayed	Pass
Score range query	Score 50–80	Players in range displayed	Pass
Update player score	ID=1 -> new score	Score updated & reordered	Pass
Count players	After insertions	Correct total count	Pass
Save data	Save option	File created (players.dat)	Pass
Load data	Restart + load	Data restored correctly	Pass
Clear data	Clear option	Tree emptied, file reset	Pass

11.2 Performance & Complexity Analysis

Performance Observations

Performance testing was conducted with varying numbers of players:

Players	Insert Time	Search Time
10	< 1 ms	< 1 ms
100	< 2 ms	< 1 ms
500	< 5 ms	< 2 ms

Observations:

- Operations remain fast even as data increases
- Red-Black Tree maintains balanced height
- No noticeable performance degradation for moderate dataset sizes

Time Complexity

Operation	Complexity
Insert	$O(\log n)$
Delete	$O(\log n)$
Search (by score/order)	$O(\log n)$
Search (by ID)	$O(n)$ (tree not ordered by ID)
Rank display (traversal)	$O(n)$
Range query	$O(n)$

Space Complexity

- Each node stores:
 - Player data (ID, name, score)
 - 3 pointers (left, right, parent)
 - Color field
- Total space complexity: $O(n)$
- File storage: $O(n)$
-

Conclusion of Testing

- All functionalities work correctly under valid and invalid inputs
- Edge cases (duplicate ID, invalid input, empty tree) are handled properly
- Red-Black Tree ensures balanced performance
- System is efficient, reliable, and scalable for moderate datasets

12. Challenges & Solutions

During the development of the Leaderboard Management System using a Red-Black Tree, several challenges were encountered. These were addressed with appropriate design and implementation strategies.

Challenge 1: Correct Implementation of Red-Black Tree Balancing

Maintaining all Red-Black Tree properties during insertion and deletion was complex, especially handling multiple fix-up cases.

Solution:

The standard CLRS-based algorithms were implemented for `fixInsert()` and `fixDelete()`. All cases involving recoloring and rotations were carefully handled and tested using different scenarios.

Challenge 2: Handling Deletion Fix-Up Cases

Deletion introduced the concept of double black, which required careful handling of multiple cases based on sibling color and children.

Solution:

The `fixDelete()` function was implemented with proper case handling:

- Red sibling case
- Black sibling with black children
- Black sibling with at least one red child

This ensured restoration of tree balance after deletion.

Challenge 3: Maintaining Unique Player IDs

Since the tree is organized by score and not ID, preventing duplicate IDs required additional validation.

Solution:

A separate function `idExists()` was implemented to check for duplicate IDs before insertion. Input validation ensures only unique IDs are accepted.

Challenge 4: Input Validation and Error Handling

Handling invalid inputs such as non-numeric IDs or scores outside the allowed range was necessary for robustness.

Solution:

- String-based input was used for ID validation
- Manual checks ensure only numeric values are accepted
- Score is restricted to the range 0–100
- Invalid inputs are rejected with proper messages

Challenge 5: Maintaining Sorted Ranking

The system needed to maintain players in descending order of scores for ranking without using extra sorting.

Solution:

The Red-Black Tree is structured based on score, and reverse in-order traversal (right -> root -> left) is used to display players in descending order.

Challenge 6: Efficient Range Queries

Displaying players within a rank range or score range required selective traversal.

Solution:

Custom traversal functions (displayRange() and displayScoreRange()) were implemented to filter and display only relevant nodes efficiently.

Challenge 7: Data Persistence Using Files

Ensuring that player data is retained after program termination was a key requirement.

Solution:

Binary file handling was implemented using fwrite() and fread():

- saveFile() stores all player records in players.dat
- loadFile() reconstructs the tree from the file

Challenge 8: Memory Management

Dynamic memory allocation required proper deallocation to avoid memory leaks.

Solution:

- freeTree() recursively frees all nodes
- clearAll() resets the tree and file
- Sentinel NIL node is handled carefully

Conclusion

All challenges were successfully resolved through structured implementation, proper validation, and adherence to Red-Black Tree properties. This resulted in a stable and efficient Leaderboard Management System.

13.2 Conclusion

The Leaderboard Management System was successfully developed using the Red-Black Tree data structure to ensure efficient data management and ranking operations.

The system supports key functionalities such as:

- Player insertion, deletion, and update
- Dynamic ranking using reverse in-order traversal
- Search and range queries
- Data persistence using binary file storage

By maintaining tree balance, the system achieves $O(\log n)$ performance for core operations like insertion and deletion. Proper input validation and error handling improve system reliability.

Overall, the project demonstrates effective use of advanced data structures to solve real-world problems involving dynamic ranking and data organization.

13.3 Future Enhancements

- The system can be further improved with the following enhancements:
- Optimized search using Hash Table
 - Maintain an additional hash map for IDs to achieve $O(1)$ search.
- Graphical User Interface (GUI)
 - Replace console-based interaction with a GUI using tools like JavaFX or web-based interfaces.
- Advanced ranking features
 - Add tie-breaking rules, player statistics, and sorting by multiple parameters.
- Multi-user and database support
 - Integrate with a database (e.g., MySQL) to support large-scale and concurrent access.
- Real-time updates and leaderboard display
 - Enable live updates and dynamic visualization of rankings.
- Performance optimization for large datasets
 - Introduce indexing or hybrid data structures for better scalability

14. References

The following resources were referred to during the development of the Leaderboard Management System:

1.Introduction to Algorithms (CLRS)

Author:- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein
– Reference for Red-Black Tree concepts and algorithms

2.Data Structures and Algorithms in C

Author :- Mark Allen Weiss
– Used for understanding tree structures and implementation logic in C

3.GeeksforGeeks – Red-Black Tree

<https://www.geeksforgeeks.org/dsa/introduction-to-red-black-tree/>

– Used for understanding insertion and deletion cases

4.JennylecturesCSIT “Red Black Tree Introduction” Youtube

<https://youtu.be/3RQtq7PDHog?si=g9fmQYJCpfGgHREP>

THANK
YOU